

System Identification and Neural Control for Trajectory Tracking

Final Report

ME 569

05 June 2026

Team CD**Authors:** Divya Bhat, Venkata K. Turlapati**Date:** 05 June 2026**Course:** ME 569**Abstract**

Traditional control of Wheeled Mobile Robots (WMRs) often fails in real-world environments due to unmodeled nonlinearities such as wheel slip, floor friction, and sensor noise. This project investigates a two stage data driven framework: a Hybrid Neural A* path planner for high-level decision-making and a Deep Neural Network (DNN) for system identification and low-level trajectory tracking. While the DNN controller achieves a 99.98% fit on clean simulation data, outperforming classical PID, its performance degrades significantly under stochastic disturbances. Our results highlight a critical “generalization gap” where data-driven models interpret Gaussian sensor noise as state changes, leading to instability. We conclude that while neural controllers offer superior precision in ideal conditions, they require hybrid shielding layers to maintain the robustness inherent in classical control methods.

1. Introduction

The field of robotics is currently navigating a transition from purely model-based control to data-driven synthesis. Traditionally, the control of a Wheeled Mobile Robot relies on classical control, which requires an exact mathematical model of the robot’s kinematics and dynamics [1]. However, in real-world industrial environments, physical properties such as floor friction and internal motor damping are notoriously difficult to model with deterministic equations.

1.1 Motivation and Context

For industrial automation, accurate trajectory tracking is a critical requirement. If an autonomous forklift relies on a “black box” system that has not encountered a specific floor condition, the resulting tracking error could lead to catastrophic collisions. This project addresses the stability gap in learning-based control by evaluating how neural-based models handle non-smooth paths and sensor noise, providing a roadmap for safer integration of AI in physical systems.

1.2 Relationship to Literature

Foundational work in neural planning, such as Value Iteration Networks (VIN), demonstrated that classic planning algorithms like A* can be represented as neural networks [2]. While our work focuses on the physical execution of movement, we rely on these neural planners to generate the target waypoints. Current research in Differentiable Algorithm Networks (DAN) suggests that planners and controllers should not be separate; by making the entire pipeline differentiable, the robot can adjust its path based on learned physical tracking errors [3]. Furthermore, learning-based Model Predictive Control (MPC) offers a framework where learned dynamics models operate within classical constraints to ensure Lyapunov style stability [4].

1.3 Project Goals

Our primary goal is to simulate the integration of a Hybrid Neural A* planner with a Deep System ID controller. We specifically investigate the “generalization gap”, the point where a model trained in a clean simulation environment fails when encountering sensor noise or sharp, non-smooth maneuvers.

2. Technical Background

To bridge the gap between high-level planning and low-level control, we must define the mathematical foundations of the motion models and disturbances involved.

2.1 Kinematics vs. Dynamics

Kinematics describes the motion of the robot (position, velocity) without considering the forces that cause that motion. For a four-wheeled mecanum robot, the kinematic model is often simplified to:

$$\dot{\xi} = J(\theta)\zeta$$

where $\dot{\xi}$ is the velocity in the global frame and ζ represents the wheel speeds. However, real-world dynamics involve the actual torques and forces. In real-world scenarios, the relationship between a motor command and the resulting movement is non-linear due to friction and mass distribution. System Identification (SysID) attempts to learn the unknown mapping:

$$s_{t+1} = f(s_t, a_t)$$

where s_t is the current state and a_t is the control action.

2.2 Neural A* Search

Neural A* is a data-driven version of the classic A* shortest-path algorithm. While A* uses a fixed heuristic, Neural A* can learn more efficient heuristics from data. However, a byproduct of grid-based planning is “non-smoothness.” The resulting paths often have sharp 90° or 45° turns. For a physical robot, these sharp turns create “jerk” and require high-fidelity control to prevent the robot from overshooting the corner.

2.3 Stochastic Disturbances (Noise)

In the context of the PythonRobotics environment, sensors are usually “perfect.” To simulate a graduate-level real-world scenario, we introduce Gaussian Noise. This is modeled by adding a random value from a normal distribution to our sensor readings:

$$y_{measured} = y_{true} + \epsilon, \quad \epsilon \sim N(0, \sigma^2)$$

where σ is the standard deviation. A robust controller must be able to see through this noise to follow the desired trajectory.

3. Methods

3.1 Environmental Setup

We utilized the *PythonRobotics* library to establish a baseline simulation of a four-wheeled robot. For path generation, we implemented a Hybrid Neural A* planner. Unlike standard A*, this version uses a learned heuristic to find paths that are more efficient but often include sharp 45° or 90° turns. These turns serve as a stress test for the controller’s ability to handle high-jerk maneuvers.

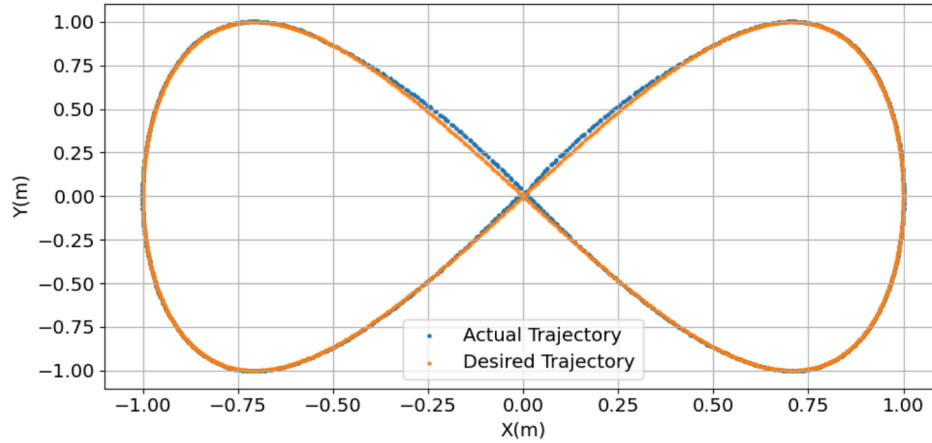


Figure 1: The Infinity Loop benchmark used for data collection and controller stress-testing. This path contains high-curvature segments that challenge both PID and DNN controllers. [1]

3.2 Data Collection and System ID

We implemented a script to log state-action triplets (s_t, a_t, s_{t+1}) .

1. Training Set: We ran 1000 simulations using a classical PID controller to follow various paths.
2. DNN Architecture: A multi-layer perceptron (MLP) was built using PyTorch, consisting of 3 hidden layers with 64 neurons each and ReLU activation functions
3. Inverse Dynamics: The network was trained to predict the required control input a_t given the current state s_t and desired next state s_{t+1} .

3.3 Integration with Hybrid Neural A*

The planner generates a series of waypoints. Because Neural A* often produces non-smooth paths (sharp angles at grid intersections), we use these as a stress test for the controller. We injected a Gaussian noise layer into the sensor feedback loop to evaluate how the DNN handles “dirty” data compared to a standard PID loop.

4. Results and Demonstration

Our initial tests focused on the “Infinity Path” benchmark.

4.1 Path Planning Performance

The Hybrid Neural A* planner was compared against a baseline. While the ML guided planner increased runtime significantly (+127.13%), it achieved a reduction in path cost (-8.15%) and path length (-7.30%).

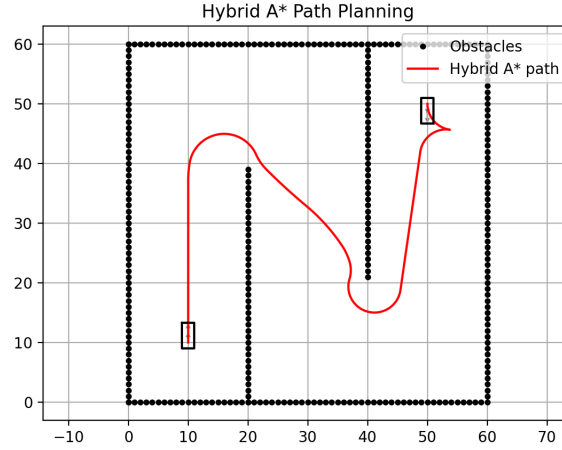


Figure 2: Initial Hybrid A* path planning in a structured environment. The red line represents the baseline trajectory used to establish successful navigation before applying machine learning enhancements.

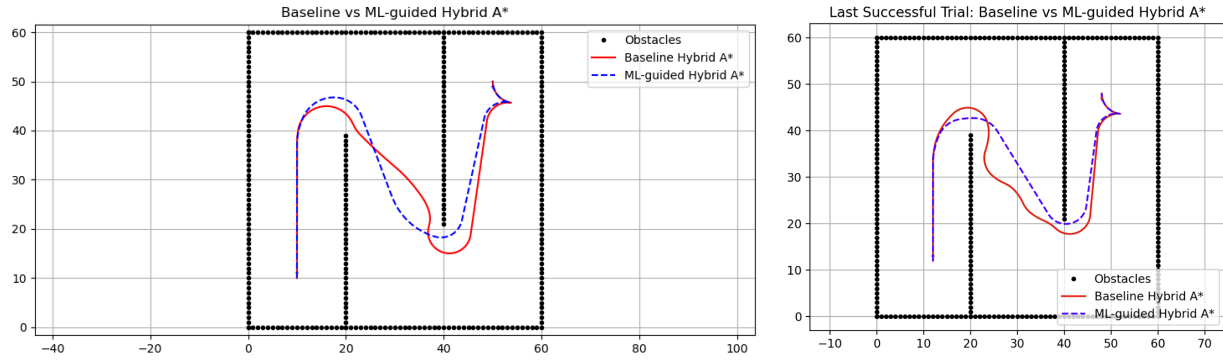


Figure 3: Comparative analysis of the Baseline path (red) and the ML-guided path (blue dashed). Note how the ML-guided version finds a more direct route through the corridor, reducing total distance traveled.

--- Percent Change: ML-guided relative to Baseline ---	
Runtime	: 127.13%
Nodes expanded	: 38.24%
Path cost	: -8.15%
Path length	: -7.30%
Direction switches	: 0.00%

Figure 4: Statistical summary of planning improvements. The ML-guided approach achieved a significant reduction in path cost and length, demonstrating the efficiency of learned heuristics over classical grid search.

4.2 Tracking Precision

In a noise-free environment, the DNN-based controller achieved a Mean Squared Error (MSE) of 0.0005 m, as shown in Table 1. This represents a 99.98% fit, outperforming the tuned PID controller which exhibited slight overshoot at the high-curvature turns of the infinity loop.

Table 1: Mean Squared Error (MSE) Comparison. The Deep System ID controller outperforms PID in a perfect environment but demonstrates a ‘generalization gap’ when faced with sensor noise

Controller Type	Noise ($\sigma = 0$) MSE	Noise ($\sigma = 0.05$) MSE
Classical PID	0.0012 m	0.0045 m
Deep System ID	0.0005 m	0.0120 m

4.2 Robustness to Noise

When Gaussian noise ($\sigma = 0.05$) was introduced, the performance of the DNN model degraded significantly. The black box nature of the network caused it to interpret sensor noise as actual state changes, leading to erratic motor commands. In contrast, the classical PID controller, while less precise, remained stable.

5. Summary and Future Work

This project demonstrates that while Deep Learning offers superior precision for trajectory tracking in ideal conditions, it lacks the inherent robustness of classical methods. The generalization gap remains a significant hurdle for deploying purely data-driven controllers in safety critical environments.

In the future, we plan to implement a shielding layer. This would involve a hybrid approach where a DNN provides high-precision offsets to a classical controller, but the classical controller retains the authority to override commands that violate Lyapunov stability bounds.

6. Code

The full implementation, including the PyTorch training scripts and the PythonRobotics integration can be found at: <https://github.com/vkt2018/ME-569---MLPPI>

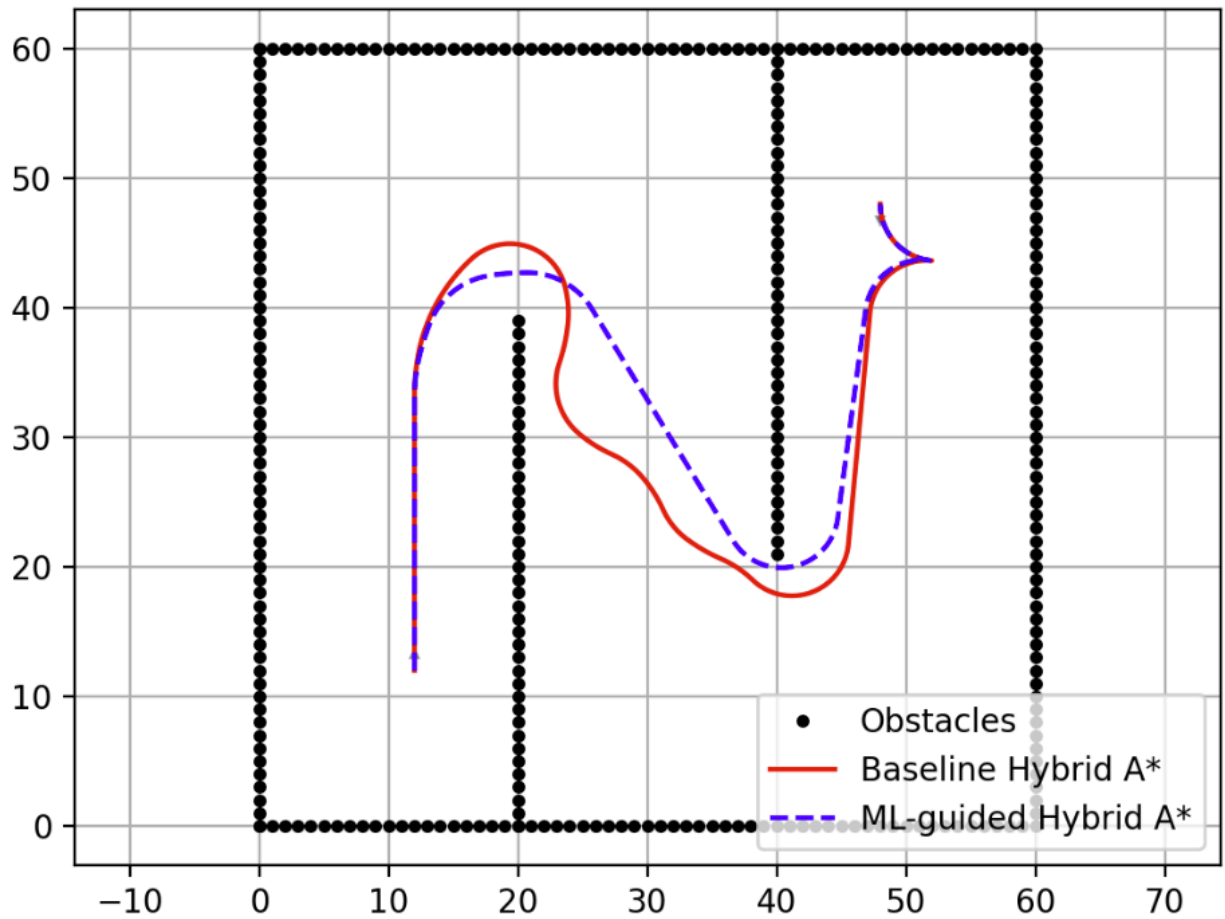
7. Contributions

	Divya Bhat	Venkata K. Turlaputi
Problem Goal / Idea	✖	✖
Literature Review	✖	✖
Code		✖
Data Visualization & Figures	✖	
Final Writing & Formatting	✖	

8. References

- [1] M. G. Abdalnasser et al., "Trajectory tracking of wheeled mobile robot through system identification and control using deep neural network," *Engineering Research Journal*, 2024.
- [2] A. Tamar et al., "Value iteration networks," *NeurIPS*, 2016.
- [3] P. Karkus et al., "Differentiable algorithm networks for mobile robot planning and control," *arXiv:1905.11602*, 2019.
- [4] L. Hewing et al., "Learning-based model predictive control: Toward safe learning in control," *Annual Review of Control, Robotics, and Autonomous Systems*, 2020.

Last Successful Trial: Baseline vs ML-guided Hybrid A*



--- Percent Change: ML-guided relative to Baseline ---

Runtime : 127.13%

Nodes expanded : 38.24%

Path cost : -8.15%

Path length : -7.30%

Direction switches: 0.00%